

Document Number: P0057R1
Date: 2015-11-06
Revises: P0057R0
Authors: Gor Nishanov <gorn@microsoft.com>
Jens Maurer <Jens.Maurer@gmx.net>
Richard Smith <richard@metafoo.co.uk>

Wording for Coroutines

Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad fomattting.

Contents

Contents	ii
List of Tables	iii
1 General	1
1.1 Scope	1
1.2 Acknowledgements	1
1.3 Normative references	1
1.4 Implementation compliance	1
1.5 Feature testing	1
1.9 Program execution	2
2 Lexical conventions	3
2.12 Keywords	3
3 Basic concepts	4
3.6 Start and termination	4
5 Expressions	5
5.3 Unary expressions	5
5.18 Assignment and compound assignment operators	6
5.21 Yield	7
6 Statements	8
6.5 Iteration statements	8
6.6 Jump statements	9
7 Declarations	10
8 Declarators	11
8.4 Function definitions	11
12 Special member functions	14
12.8 Copying and moving class objects	14
13 Overloading	15
13.5 Overloaded operators	15
18 Language support library	16
18.1 General	16
18.10 Other runtime support	16
18.11 Coroutines support library	16

List of Tables

1	Feature-test macro	1
2	Language support library summary	16

1 General

[intro]

1.1 Scope

[intro.scope]

- ¹ This Technical Specification describes extensions to the C++ Programming Language (1.3) that enable definition of coroutines. These extensions include new syntactic forms and modifications to existing language semantics.
- ² The International Standard, ISO/IEC 14882, provides important context and specification for this Technical Specification. This document is written as a set of changes against that specification. Instructions to modify or add paragraphs are written as explicit instructions. Modifications made directly to existing text from the International Standard use underlining to represent added text and ~~strikethrough~~ to represent deleted text.

1.2 Acknowledgements

[intro.ack]

This work is the result of collaboration of researchers in industry and academia, including CppDes Microsoft group and the WG21 study group SG1. We wish to thank people who made valuable contributions within and outside these groups, including Artur Laksberg, Richard Smith, Chandler Carruth, Gabriel Dos Reis, Deon Brewis, Jonathan Caves, James McNellis, David Vandevor, Stephan T. Lavavej, Herb Sutter, Pablo Halpern, Robert Schumacher, Michael Wong, Niklas Gustafsson, Nick Maliwacki, Vladimir Petter, Shahms King, Slava Kuznetsov, Tongari J, Lewis Baker, Lawrence Cowl, and many others not named here who contributed to the discussion.

1.3 Normative references

[intro.refs]

- ¹ The following referenced document is indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.
- (1.1) — ISO/IEC 14882:2014, *Programming Languages – C++*

ISO/IEC 14882:2014 is hereafter called the *C++ Standard*. Beginning with section 1.9 below, all clause and section numbers, titles, and symbolic references in [brackets] refer to the corresponding elements of the C++ Standard. Sections 1.1 through 1.5 of this Technical Specification are introductory material and are unrelated to the similarly-numbered sections of the C++ Standard.

1.4 Implementation compliance

[intro.compliance]

- ¹ Conformance requirements for this specification are the same as those defined in section 1.4 of the C++ Standard. [*Note: Conformance is defined in terms of the behavior of programs. — end note*]

1.5 Feature testing

[intro.features]

An implementation that provides support for this Technical Specification shall define the feature test macro in Table 1.

Table 1 — Feature-test macro

Name	Value	Header
<code>__cpp_coroutines</code>	201510	<i>predeclared</i>

1.9 Program execution

[intro.execution]

Modify paragraph 7 to read:

- ⁷ An instance of each object with automatic storage duration (3.7.3) is associated with each entry into its block. Such an object exists and retains its last-stored value during the execution of the block and while the block is suspended (by a call of a function, [suspension of a coroutine \(5.3.8\)](#), or receipt of a signal).

2 Lexical conventions

[lex]

2.12 Keywords

[lex.key]

Add the keywords `co_await`, `co_yield`, and `co_return` to Table 4 "Keywords".

3 Basic concepts

[basic]

3.6 Start and termination

[basic.start]

3.6.1 Main function

[basic.start.main]

Add underlined text to paragraph 3.

- ³ The function `main` shall not be used within a program. The linkage (3.5) of `main` is implementation-defined. A program that defines `main` as deleted or that declares `main` to be `inline`, `static`, or `constexpr` is ill-formed. The function `main` shall not be a coroutine (8.4.4). The name `main` is not otherwise reserved. [*Example:* member functions, classes, and enumerations can be called `main`, as can entities in other namespaces. — *end example*]

5 Expressions

[expr]

5.3 Unary expressions

[expr.unary]

Add *await-expression* to the grammar production *unary-expression*:

```

unary-expression:
    postfix-expression
    ++ cast-expression
    -- cast-expression
    await-expression
    unary-operator cast-expression
    sizeof unary-expression
    sizeof ( type-id )
    sizeof ... ( identifier )
    alignof ( type-id )
    noexcept-expression
    new-expression
    delete-expression

```

5.3.8 Await

[expr.await]

Add this section to 5.3.

- ¹ The `co_await` operator is used to suspend evaluation of a coroutine (8.4.4) while awaiting completion of the computation represented by the operand expression.

```

await-expression:
    co_await cast-expression

```

- ² An *await-expression* shall only appear in a potentially evaluated expression within the *compound-statement* of a *function-body* outside of a *handler* (15). In a *declaration-statement* or in the *simple-declaration* (if any) of a *for-init-statement*, an *await-expression* shall only appear in an *initializer* of that *declaration-statement* or *simple-declaration*. An *await-expression* shall not appear in a default argument (8.3.6). A context within a function where an *await-expression* can appear is called a *suspension context* of the function.

- ³ Evaluation of an *await-expression* involves the following auxiliary expressions:

- (3.1) — If the *cast-expression* is a prvalue, *a* is a temporary object copy-initialized from the *cast-expression*, otherwise *a* is an lvalue referring to the result of evaluating the *cast-expression*.
- (3.2) — *p* is an lvalue naming the promise object (8.4.4) of the enclosing coroutine and *P* is a type of that object.
- (3.3) — The *unqualified-id* `await_transform` is looked up within the scope of *P* as if by class member access lookup (3.4.5), and if this lookup finds at least one declaration, then *e* is `p.await_transform(a)`; otherwise, *e* is *a*.
- (3.4) — *h* is an object of type `std::coroutine_handle<P>` referring to the enclosing coroutine.
- (3.5) — *await_ready* is the expression `e.await_ready()`, contextually converted to `bool`.
- (3.6) — *await_suspend* is the expression `e.await_suspend(h)`, which shall be a prvalue of type `void` or `bool`.
- (3.7) — *await_resume* is the expression `e.await_resume()`.

- 4 The *await-expression* has the same type and value category as the **await-resume** expression.
- 5 The *await-expression* evaluates the *await-ready* expression, then:
- (5.1) — If the result is **true**, a coroutine is considered suspended. Then, the *await-suspend* expression is evaluated. If that expression has type **bool** and returns **false**, the coroutine is resumed. If that expression exits via an exception, the exception is caught, the coroutine is resumed, and the exception is immediately re-thrown (15.1). Otherwise, control flow returns to the current caller or resumer (8.4.4).
 - (5.2) — If the result is **false**, or when the coroutine is resumed, the *await-resume* expression is evaluated, and its result is the result of the *await-expression*.

6 [*Example:*

```
template <typename T>
struct my_future {
    ...
    bool await_ready();
    void await_suspend(std::coroutine_handle<>);
    T await_resume();
};

template <class Rep, class Period>
auto operator co_await(std::chrono::duration<Rep, Period> d) {
    struct awaiter {
        std::chrono::system_clock::duration duration;
        ...
        awaiter(std::chrono::system_clock::duration d) : duration(d){}
        bool await_ready() const { return duration.count() <= 0; }
        void await_resume() {}
        void await_suspend(std::coroutine_handle<> h){...}
    };
    return awaiter{d};
}

using namespace std::chrono;

my_future<void> g() {
    std::cout << "just about go to sleep...\n";
    co_await 10ms;
    std::cout << "resumed\n";
}
```

— *end example*]

5.18 Assignment and compound assignment operators

[**expr.ass**]

Add *yield-expression* to the grammar production *assignment-expression*.

assignment-expression:

conditional-expression

logical-or-expression *assignment-operator* *initializer-clause*

throw-expression

yield-expression

5.21 Yield

[expr.yield]

Add a new section to Clause 5.

yield-expression:
 co_yield *assignment-expression*
 co_yield *braced-init-list*

- ¹ A *yield-expression* shall only appear within a suspension context of a function (5.3.8). Let *e* be the operand of the *yield-expression* and *p* be an lvalue naming the promise object of the enclosing coroutine (8.4.4), then the *yield-expression* is equivalent to the expression `co_await p.yield_value(e)`.

[*Example*:

```
template <typename T>
struct my_generator {
    struct promise_type {
        T current_value;
        ...
        auto yield_value(T v) {
            current_value = std::move(v);
            return std::suspend_always{};
        }
    };
    struct iterator { ... };
    iterator begin();
    iterator end();
};

my_generator<pair<int,int>> g1() {
    for (int i = 0; i < 10; ++i) co_yield {i,i};
}
my_generator<pair<int,int>> g2() {
    for (int i = 0; i < 10; ++i) co_yield make_pair(i,i);
}

auto f(int x = co_yield 5); // error: yield-expression outside of function suspension context
int a[] = { co_yield 1 }; // error: yield-expression outside of function suspension context

int main() {
    auto r1 = g1();
    auto r2 = g2();
    assert(std::equal(r1.begin(), r1.end(), r2.begin(), r2.end()));
}
```

— end example]

6 Statements

[stmt.stmt]

6.5 Iteration statements

[stmt.iter]

Add underlined text to paragraph 1.

- ¹ Iteration statements specify looping.

```
iteration-statement:
    while ( condition ) statement
    do statement while ( expression ) ;
    for ( for-init-statement conditionopt; expressionopt ) statement
    for co_awaitopt ( for-range-declaration : for-range-initializer ) statement
```

6.5.4 The range-based for statement

[stmt.ranged]

Add the underlined text to paragraph 1.

- ¹ For a range-based **for** statement of the form

```
for co_awaitopt ( for-range-declaration : expression ) statement
```

let *range-init* be equivalent to the *expression* surrounded by parentheses¹

```
( expression )
```

and for a range-based **for** statement of the form

```
for co_awaitopt ( for-range-declaration : braced-init-list ) statement
```

let *range-init* be equivalent to the *braced-init-list*. In each case, a range-based **for** statement is equivalent to

```
{
    auto && __range = range-init;
    for ( auto __begin = co_awaitopt begin-expr,
        __end = end-expr;
        __begin != __end;
        co_awaitopt ++__begin ) {
        for-range-declaration = *__begin;
        statement
    }
}
```

where co_await is present only if it appeared immediately after the **for** keyword, and __range, __begin, and __end are variables defined for exposition only, and _RangeT is the type of the expression, and *begin-expr* and *end-expr* are determined as follows: ...

Add the following paragraph after paragraph 2.

- ³ A range-based **for** statement with **co_await** shall only appear within a suspension context of a function (5.3.8).

¹) this ensures that a top-level comma operator cannot be reinterpreted as a delimiter between *init-declarators* in the declaration of **__range**.

6.6 Jump statements

[stmt.jump]

In paragraph 1 add two productions to the grammar:

```

jump-statement:
    break ;
    continue ;
    return expressionopt;
    return braced-init-list ;
    coroutine-return-statement
    goto identifier ;

```

6.6.3 The return statement

[stmt.return]

Add underlined text to paragraph 1:

- 1 A function returns to its caller by the **return** statement; that function shall not be a coroutine (8.4.4).

6.6.3.1 The co_return statement

[stmt.return.coroutine]

Add this section to 6.6.

```

coroutine-return-statement:
    co_return expressionopt;
    co_return braced-init-list;

```

- 1 A coroutine returns to its caller by the **co_return** statement or when suspended (5.3.8).
- 2 The *expression* or *braced-init-list* of a **co_return** statement is called its operand. If the operand is an expression of non-void type, it is considered to be an xvalue. Let *P* be the coroutine promise type (8.4.4) and *p* be an lvalue naming the coroutine promise object (8.4.4), then a **co_return** statement is equivalent to:

```
{ S; goto final_suspend_label; }
```

where *final_suspend_label* is as defined in 8.4.4 and *S* is an expression defined as follows:

- (2.1) — *S* is *p.return_value*(*braced-init-list*), if the operand is *braced-init-list*;
- (2.2) — *S* is *p.return_value*(*expression*), if the operand is an expression of non-void type;
- (2.3) — *S* is *p.return_void*(), otherwise;

S shall be a prvalue of type void.

- 3 The *unqualified-ids* **return_void** and **return_value** are looked up in the scope of class *P*. If both are found, the program is ill-formed. Flowing off the end of a coroutine is equivalent to a **co_return** with no operand; this results in undefined behavior if expression *p.return_void*() is ill-formed.

7 Declarations

[dcl.dcl]

7.1.5 The `constexpr` specifier

[dcl.constexpr]

Insert a new bullet after paragraph 3 bullet 1.

- 3 The definition of a `constexpr` function shall satisfy the following constraints:
- (3.1) — it shall not be virtual (10.3);
 - (3.2) — [it shall not be a coroutine \(8.4.4\)](#);
 - (3.3) — ...

7.1.6.4 `auto` specifier

[dcl.spec.auto]

Add the following paragraph.

- 15 A function declared with a return type that uses a placeholder type shall not be a coroutine [\(8.4.4\)](#).

8 Declarators

[dcl.decl]

8.4 Function definitions

[dcl.fct.def]

8.4.4 Coroutines

[dcl.fct.def.coroutine]

Add this section to 8.4.

¹ A function is a *coroutine* if it contains a *coroutine-return-statement* (6.6.3.1), an *await-expression* (5.3.8), a *yield-expression* (5.21), or a *range-based-for* (6.5.4) with `co_await`. The *parameter-declaration-clause* of the coroutine shall not terminate with an ellipsis that is not part of an *abstract-declarator*.

² [Example:

```
task<int> f();

task<void> g1() {
    int i = co_await f();
    std::cout << "f() => " << i << std::endl;
}

template <typename... Args>
task<void> g2(Args&&...) { // OK: ellipsis is a pack expansion
    int i = co_await f();
    std::cout << "f() => " << i << std::endl;
}

task<void> g3(int a, ...) { // error: variable parameter list not allowed
    int i = co_await f();
    std::cout << "f() => " << i << std::endl;
}
```

— end example]

³ For a coroutine f that is a non-static member function, let P_1 denote the type of the implicit object parameter (13.3.1) and $P_2 \dots P_n$ be the types of the function parameters; otherwise let $P_1 \dots P_n$ be the types of the function parameters. Let R be the return type and F be the *function-body* of f , T be the type `std::coroutine_traits< $R, P_1, \dots, P_n$ >`, and P be the class type denoted by `T::promise_type`. Then, the coroutine behaves as if its body were:

```
{
    P p;
    co_await p.initial_suspend(); // initial suspend point
    F'
    final_suspend:
    co_await p.final_suspend(); // final suspend point
}
```

where F' is F unless the *unqualified-id* `set_exception` is found in the scope of P as if by class member access lookup (3.4.5), then F' is

```
try { F } catch(...) { p.set_exception(std::current_exception()); }
```

No header needs to be included for this use of the `std::current_exception`. An object denoted as *p* is the *promise object* of the coroutine and type *P* is a *promise type* of the coroutine.

When a coroutine returns to its caller, the return value is obtained by a call to `p.get_return_object()`. A call to a `get_return_object` is sequenced before the call to `initial_suspend`.

A suspended coroutine can be resumed to continue execution by invoking a resumption member function (18.11.2.4) of an object of type `coroutine_handle<P>` associated with this instance of the coroutine. The function that invoked a resumption member function is called *resumer*. Invoking a resumption member function for a coroutine that is not suspended results in undefined behavior.

A *coroutine state* consists of storage for objects with automatic storage duration of a coroutine. An implementation may need to allocate memory for coroutine state. If so, it shall obtain the storage by calling an *allocation function* (3.7.4.1). The allocation function's name is looked up in the scope of the promise type of the coroutine. If this lookup fails to find the name, the allocation function's name is looked up in the global scope. If the lookup finds an allocation function that takes exactly one parameter, it will be used, otherwise, all parameters of the coroutine are passed to the allocation function after the size parameter in order.

The coroutine state is destroyed when the control flows off the end of the coroutine or the `destroy` member function (18.11.2.4) of an object of `std::coroutine_handle<P>` associated with this coroutine is invoked. In the latter case objects with automatic storage duration that are in scope at the suspend point are destroyed in the reverse order of the construction. If the coroutine state required dynamic allocation, the storage is released by calling a deallocation function (3.7.4.2). If `destroy` is called for a coroutine that is not suspended, the program has undefined behavior.

The deallocation function's name is looked up in the scope of the coroutine promise type. If this lookup fails to find the name, the deallocation function's name is looked up in the global scope. If deallocation function lookup finds both a usual deallocation function with only a pointer parameter and a usual deallocation function with both a pointer parameter and a size parameter, then the selected deallocation function shall be the one with two parameters. Otherwise, the selected deallocation function shall be the function with one parameter.

When a coroutine is invoked, each of its parameters is moved to the coroutine state, as if parameters were members of a class and were moved by defaulted move constructor as specified in 12.8. A reference to a parameter in the function-body of the coroutine is replaced by a reference to the copy of the parameter.

If the unqualified-id `get_return_object_on_allocation_failure` is looked up in the scope of class *P* as if by class member access lookup (3.4.5), and if a declaration was found, then `std::nothrow_t` forms of allocation and deallocation functions will be used. If an allocation function returned `nullptr`, the coroutine must return control to the caller of the coroutine and the return value shall be obtained by a call to `P::get_return_object_on_allocation_failure()`.

[Example:

```
// using nothrow operator new
struct generator {
    using handle = std::coroutine_handle<promise_type>;
    struct promise_type {
        int current_value;
        static auto get_return_object_on_allocation_failure() { return generator{nullptr}; }
        auto get_return_object() { return generator{handle::from_promise(*this)}; }
        auto initial_suspend() { return std::suspend_always{}; }
        auto final_suspend() { return std::suspend_always{}; }
        auto yield_value(int value) {
```

```

        current_value = value;
        return std::suspend_always{};
    }
};
bool move_next() { return coro ? (coro.resume(), !coro.done()) : false; }
int current_value() { return coro.promise().current_value; }
~generator() { if(coro) coro.destroy(); }
private:
    generator(handle h) : coro(h) {}
    handle coro;
};
generator f() { co_yield 1; co_yield 2; }

int main() {
    auto g = f();
    while (g.move_next()) std::cout << g.current_value() << std::endl;
}

```

— *end example*]

11

[*Example:*

```

// using a stateful allocator
class Arena;
struct my_coroutine {
    struct promise_type {
        ...
        template <typename... TheRest>
        void* operator new(std::size_t size, Arena& pool, TheRest const&...) {
            return pool.allocate(size);
        }
        void* operator delete(void* p, std::size_t size) {
            return pool.deallocate(p, size);
        }
    };
};

my_coroutine (Arena& a) {
    // will call my_coroutine::promise_type::operator new(<required-size>, a)
    // to obtain storage for the coroutine state
    co_yield 1;
}

int main() {
    Pool memPool;
    for (int i = 0; i < 1'000'000; ++i) my_coroutine(memPool);
}

```

— *end example*]

12 Special member functions [special]

Add new paragraph after paragraph 5.

- 6 A special member function shall not be a coroutine.

12.8 Copying and moving class objects [class.copy]

Add the underlined text to paragraph 31.

- 31 When certain criteria are met, an implementation is allowed to omit the copy/move construction of a class object, even if the constructor selected for the copy/move operation and/or the destructor for the object have side effects. In such cases, the implementation treats the source and target of the omitted copy/move operation as simply two different ways of referring to the same object, and the destruction of that object occurs at the later of the times when the two objects would have been destroyed without the optimization.² This elision of copy/move operations, called *copy elision*, is permitted in the following circumstances (which may be combined to eliminate multiple copies):
- (31.1) — in a **return** statement in a function with a class return type, when the expression is the name of a non-volatile automatic object (other than a function or catch-clause parameter) with the same cv-unqualified type as the function return type, the copy/move operation can be omitted by constructing the automatic object directly into the function's return value
 - (31.2) — When a parameter would be copied/moved to the coroutine state (8.4.4) the copy/move can be omitted by continuing to refer to the function parameters instead of referring to their copies in the coroutine state.

2) Because only one object is destroyed instead of two, and one copy/move constructor is not executed, there is still one object destroyed for each one constructed.

13 Overloading

[over]

13.5 Overloaded operators

[over.oper]

Add `co_await` to the list of operators in paragraph 1 before operators `()` and `[]`.

13.3.1.2 Operators in expressions

[over.match.oper]

Change 13.3.1.2/9:

- 9 If the operator is the operator `,`, the unary operator `&`, ~~or~~ the operator `->`, or the operator `co_await`, and there are no viable functions, then the operator is assumed to be the built-in operator and interpreted according to Clause 5.

Add a new paragraph after paragraph 8:

- 9 When operator `co_await` returns, the `co_await` operator is applied to the value returned. The resulting `co_await` operator is assumed to be the built-in operator and interpreted according to Clause 5.

18 Language support library

[language.support]

18.1 General

[support.general]

Add a row to Table 2 for coroutine support header `<coroutine>`.

Table 2 — Language support library summary

Subclause	Header(s)
18.2 Types	<code><cstddef></code>
	<code><limits></code>
18.3 Implementation properties	<code><climits></code>
	<code><cfloat></code>
18.4 Integer types	<code><stdint></code>
18.5 Start and termination	<code><cstdlib></code>
18.6 Dynamic memory management	<code><new></code>
18.7 Type identification	<code><typeinfo></code>
18.8 Exception handling	<code><exception></code>
18.9 Initializer lists	<code><initializer_list></code>
18.11 Coroutines support	<code><coroutine></code>
	<code><csignal></code>
	<code><csetjmp></code>
	<code><cstdalign></code>
18.10 Other runtime support	<code><cstdarg></code>
	<code><cstdbool></code>
	<code><cstdlib></code>
	<code><ctime></code>

18.10 Other runtime support

[support.runtime]

Add underlined text to paragraph 4.

- ⁴ The function signature `longjmp(jmp_buf jbuf, int val)` has more restricted behavior in this International Standard. A `setjmp/longjmp` call pair has undefined behavior if replacing the `setjmp` and `longjmp` by `catch` and `throw` would invoke any non-trivial destructors for any automatic objects. A call to `setjmp` or `longjmp` has undefined behavior if invoked in a coroutine.
- SEE ALSO: ISO C 7.10.4, 7.8, 7.6, 7.12.

18.11 Coroutines support library

[support.coroutine]

Add this section to clause 18.

- ¹ The header `<coroutine>` defines several types providing compile and run-time support for coroutines in a C++ program.

Header `<coroutine>` synopsis

```
namespace std {
    inline namespace coroutines_v1 {
```

```

// 18.11.1 coroutine traits
template <typename R, typename... ArgTypes>
    class coroutine_traits;

// 18.11.2 coroutine handle
template <typename Promise = void>
    class coroutine_handle;

// 18.11.2.7 comparison operators:
bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;
bool operator<(coroutine_handle<> x, coroutine_handle<> y) noexcept;
bool operator!=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
bool operator<=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
bool operator>=(coroutine_handle<> x, coroutine_handle<> y) noexcept;
bool operator>(coroutine_handle<> x, coroutine_handle<> y) noexcept;

// 18.11.3 trivial awaitables
struct suspend_never;
struct suspend_always;

} // namespace coroutines_v1

// 18.11.2.8 hash support:
template <class T> struct hash;
template <class P> struct hash<coroutine_handle<P>>;
} // namespace std

```

18.11.1 coroutine traits [coroutine.traits]

- 1 This subclause defines requirements on classes representing *coroutine traits*, and defines the class template `coroutine_traits` that satisfies those requirements.
- 2 The `coroutine_traits` may be specialized by the user to customize the semantics of coroutines.

18.11.1.1 Struct template `coroutine_traits` [coroutine.traits.primary]

- 1 The header `<coroutine>` shall define the class template `coroutine_traits` as follows:

```

namespace std {
    inline namespace coroutines_v1 {
        template <typename R, typename... Args>
        struct coroutine_traits {
            using promise_type = typename R::promise_type;
        };
    } // namespace coroutines_v1
} // namespace std

```

18.11.2 Struct template `coroutine_handle` [coroutine.handle]

```

namespace std {
    inline namespace coroutines_v1 {
        template <>
        struct coroutine_handle<void>
        {
            // 18.11.2.1 construct/reset
            constexpr coroutine_handle() noexcept;
            constexpr coroutine_handle(nullptr_t) noexcept;

```

```

coroutine_handle& operator=(nullptr_t) noexcept;

// 18.11.2.2 export/import
void* address() const noexcept;
static coroutine_handle from_address(void* addr) noexcept;

// 18.11.2.3 capacity
explicit operator bool() const noexcept;

// 18.11.2.4 resumption
void operator()() const;
void resume() const;
void destroy() const;

// 18.11.2.5 completion check
bool done() const noexcept;
};

template <typename Promise>
struct coroutine_handle : coroutine_handle<>
{
    // 18.11.2.1 construct/reset
    using coroutine_handle<>::coroutine_handle;
    static coroutine_handle from_promise(Promise&) noexcept;
    coroutine_handle& operator=(nullptr_t) noexcept;

    // 18.11.2.6 promise access
    Promise& promise() const noexcept;
};
} // namespace coroutines_v1
} // namespace std

```

- 1 Let P be a promise type of the coroutine (8.4.4). An object of the type `coroutine_handle<P>` is called a *coroutine handle* and can be used to refer to a suspended or executing coroutine. A default constructed `coroutine_handle` object does not refer to any coroutine.

18.11.2.1 `coroutine_handle` construct/reset [coroutine.handle.con]

- ```

constexpr coroutine_handle() noexcept;
constexpr coroutine_handle(nullptr_t) noexcept;

```
- 1 *Postconditions:* `address() == nullptr`.
- ```

static coroutine_handle::from_promise(Promise& p) noexcept;

```
- 2 *Requires:* `p` is a reference to a promise object of a coroutine.
- 3 *Postconditions:* `address() != nullptr` and `addressof(promise()) == addressof(p)`.
- ```

coroutine_handle& operator=(nullptr_t) noexcept;

```
- 4 *Postconditions:* `address() == nullptr`.
- 5 *Returns:* `*this`.

#### 18.11.2.2 `coroutine_handle` export/import [coroutine.handle.export]

- ```

static coroutine_handle from_address(void* addr) noexcept;
void* address() const noexcept;

```
- 1 *Postconditions:* `coroutine_handle<>::from_address(address()) == *this`.

18.11.2.3 coroutine_handle capacity [coroutine.handle.capacity]

explicit operator bool() const noexcept;

1 *Returns:* true if address() != nullptr, otherwise false.

18.11.2.4 coroutine_handle resumption [coroutine.handle.resumption]

void operator()() const;

void resume() const;

1 *Requires:* *this refers to a suspended coroutine.

2 *Effects:* resumes the execution of the coroutine. If the coroutine was suspended at the final suspend point, behavior is undefined.

void destroy() const;

3 *Requires:* *this refers to a suspended coroutine.

4 *Effects:* destroys the coroutine (8.4.4).

18.11.2.5 coroutine_handle completion check [coroutine.handle.completion]

bool done() const noexcept;

1 *Requires:* *this refers to a suspended coroutine.

2 *Returns:* true if the coroutine is suspended at final suspend point, otherwise false.

18.11.2.6 coroutine_handle promise access [coroutine.handle.prom]

Promise& promise() noexcept;

Promise const& promise() const noexcept;

1 *Requires:* *this refers to a coroutine.

2 *Returns:* a reference to a promise of the coroutine.

18.11.2.7 Comparison operators [coroutine.handle.compare]

bool operator==(coroutine_handle<> x, coroutine_handle<> y) noexcept;

1 *Returns:* x.address() == y.address().

bool operator<(coroutine_handle<> x, coroutine_handle<> y) noexcept;

2 *Returns:* less<void*>()(x.address(), y.address()).

bool operator!=(coroutine_handle<> x, coroutine_handle<> y) noexcept;

3 *Returns:* !(x == y).

bool operator>(coroutine_handle<> x, coroutine_handle<> y) noexcept;

4 *Returns:* (y < x).

bool operator<=(coroutine_handle<> x, coroutine_handle<> y) noexcept;

5 *Returns:* !(x > y).

bool operator>=(coroutine_handle<> x, coroutine_handle<> y) noexcept;

6 *Returns:* !(x < y).

18.11.2.8 Hash support [coroutine.handle.hash]

template <class P> struct hash<coroutine_handle<P>>;

1 The template specializations shall meet the requirements of class template hash (20.9.12).

18.11.3 trivial awaitables**[coroutine.trivial.awaitables]**

The header `<coroutine>` shall define `suspend_never` and `suspend_always` as follows.

```
struct suspend_never {
    bool await_ready() { return true; }
    void await_suspend(coroutine_handle<>) {}
    void await_resume() {}
};
struct suspend_always {
    bool await_ready() { return false; }
    void await_suspend(coroutine_handle<>) {}
    void await_resume() {}
};
```